Kasi Yedumati · Full Stack Training

Lesson 7 — Project Structure

Course: FastAPI for Full Stack · **Session 7 of 10** · Learning path · **Previous:** Lesson 6 · **Next:** Lesson 8

> **Module Outcome:** You can put the router, the service, and the repository in their own files, hand each route a service with `Depends`, read the database URL from the environment, and run one automated test.

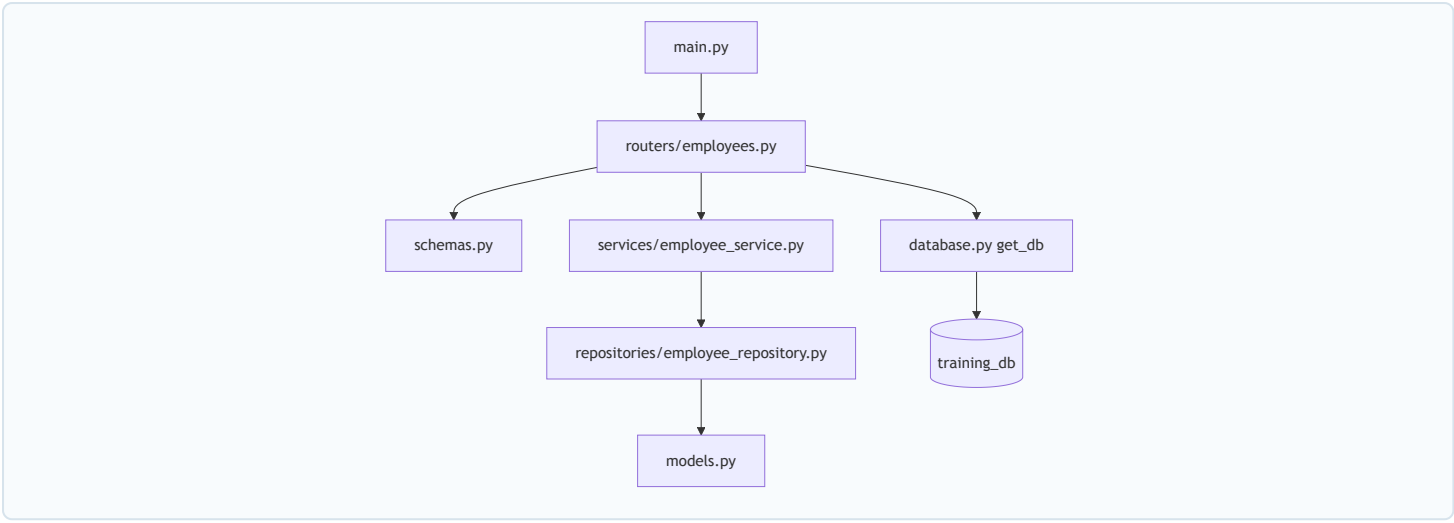
You will build	The employee API as a small project
New words	Router, dependency, environment variable
You need	The employee CRUD API from Lesson 6, with `EmployeeRepository` and `EmployeeService`

From the requirements. One `main.py` that holds the table, the checks, the queries, the rules, and every route is hard to change once login and a web page arrive. The layers from Lessons 5 and 6 already separate those jobs. Now each layer gets its own file. The behaviour of the API stays the same.

1. One layer per file

```
acmehr-api/
├── main.py                      ← creates the app, the exception handlers, and the seed
├── database.py                  ← URL, engine, session, get_db
├── models.py                    ← Employee table
├── schemas.py                   ← EmployeeCreate, EmployeeUpdate, EmployeeOut
├── repositories/
│   └── employee_repository.py   ← EmployeeRepository: the queries
├── services/
│   └── employee_service.py      ← EmployeeService, EmployeeNotFound, DuplicateEmployeeId
├── routers/
│   └── employees.py             ← the routes
├── .env                         ← your machine. Not committed
├── .env.example                 ← the names of the settings, committed
├── .gitignore                   ← keeps .env, venv/, and test databases out of Git
├── requirements.txt
└── test_employees.py
```

`repositories/`, `services/`, and `routers/` each hold an empty `__init__.py`. The same files are in `examples/lesson07/`.



Each arrow is an `import`. The router imports the service, the service imports the repository, and the repository imports the table. Nothing imports upwards: the repository never imports the service, and the service never imports FastAPI.

The classes move as they are. `EmployeeRepository` and `EmployeeService` are the Lesson 6 code with imports added at the top:

```
# services/employee_service.py
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from repositories.employee_repository import EmployeeRepository
```

`database.py` is where the URL lives. Copy `.env.example` to `.env` and put your MySQL password in `.env` if it is not `password123`.

```
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

load_dotenv()

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "mysql+mysqlconnector://trainee:password123@localhost:3306/training_db",
)
connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
engine = create_engine(DATABASE_URL, connect_args=connect_args)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`yield` hands the session over and continues after the route finishes, which is where `close()` runs.

---

2. `Depends` hands each route a service

In Lesson 6 every route opened `SessionLocal()`, built `EmployeeService(db)`, and closed the session in `finally`. Six routes, six copies. One small function in the router file replaces them:

```
from fastapi import APIRouter, Depends, Response
from sqlalchemy.orm import Session

from database import get_db
from services.employee_service import EmployeeService

router = APIRouter(prefix="/employees", tags=["employees"])

def get_employee_service(db: Session = Depends(get_db)) -> EmployeeService:
    return EmployeeService(db)

@router.get("", response_model=list[EmployeeOut])
def list_employees(
    department: str | None = None,
    service: EmployeeService = Depends(get_employee_service),
):
    return service.get_all(department)

@router.post("", status_code=201, response_model=EmployeeOut)
def create_employee(employee: EmployeeCreate, service: EmployeeService = Depends(get_employee_service)):
    return service.register(employee)
```

Piece	Job
`APIRouter`	A group of routes. `prefix` adds `/employees` so the path here is `""`
`tags`	The heading on `/docs`
`Depends(get_db)`	FastAPI calls `get_db`, passes the session on, and closes it after the response
`get_employee_service`	Builds one `EmployeeService` on that session. It is a dependency itself
`Depends(get_employee_service)`	The route asks for a service. It never sees `SessionLocal`
`include_router`	In `main.py`: `app.include_router(employees_router)`

`get_employee_service` lives in the router file, not in the service file. `Depends` is FastAPI. The service stays plain Python, so a test or a script can build `EmployeeService(db)` without a request.

PUT, PATCH, GET by id, and DELETE follow the same shape: one line that calls the service. The whole file is `examples/lesson07/routers/employees.py`.

`main.py` keeps what belongs to the whole app: `SEED`, `lifespan` with `EmployeeService(db).seed(SEED)`, `app.include_router(employees_router)`, and the two exception handlers. A router cannot register an exception handler, so they stay on `app`:

```
from services.employee_service import DuplicateEmployeeId, EmployeeNotFound, EmployeeService

@app.exception_handler(EmployeeNotFound)
```

```
def employee_not_found(request: Request, exc: EmployeeNotFound):
    return JSONResponse(status_code=404, content={"detail": "Employee not found"})
```

`pip install python-dotenv` so `load_dotenv()` reads `.env`. `.env` stays off GitHub. `.env.example` is the file you commit.

From this folder:

```
fastapi dev main.py
```

`/docs` still shows the employee routes. GET Ananya still returns **200**. A duplicate POST is still **400**.

---

3. One test

`test_employees.py` points `DATABASE_URL` at a SQLite file before it imports the app, then calls the same routes. You do not need MySQL to see the test pass. The class app still uses MySQL when you run `fastapi dev`.

```
pip install pytest
pytest test_employees.py
```

Output:

```
3 passed
```

The tests check `/health`, Ananya's name, and a **404** for `E-9999`.

Common error: `ModuleNotFoundError: No module named 'database'` or `'services'`. The terminal is not in the folder that contains `main.py`. `cd` there, then `fastapi dev main.py`.

Common error: `ImportError: cannot import name 'EmployeeService' from partially initialized module`. Two files import each other. Check the arrows in section 1: the repository must not import the service, and the service must not import the router.

Common error: You committed `.env`. It holds the database password. The example folder already has a `.gitignore` that lists `.env`. Copy it into your project before your first commit.

Try these

1. **(Easy)** Open `/docs`. The employee routes sit under the tag **employees**.
2. **(Easy)** Run `pytest`. Three tests pass.
3. **(Medium)** Move `DATABASE_URL` only in `.env`, restart, and GET Ananya. The route file did not change.
4. **(Medium)** Search the project for `HTTPException`, `commit`, and `query`. Each word appears in one layer only. Name the layer.
5. **(Stretch)** Add `GET /employees/count` that returns `{"count": n}`. Add `count` to the service, which calls `repo.count()`. Declare the route above `/{employee_id}`, or `count` is read as an employee id.

Interview: *What problem does Depends solve?* — The route asks for a service and does not open or close the session by hand. Every route gets it the same way, and the session is closed after every response.

Interview: *Why does `get_employee_service` live in the router and not in the service?* — `Depends` is FastAPI. Keeping it out of the service means the service runs the same in a route, in a script, and in a test.

Key takeaways

- Router, service, and repository each have a file. Imports only point downwards.
- `APIRouter` groups the employee routes. `main.py` includes that router and registers the exception handlers.
- `Depends(get_employee_service)` builds a service on a session from `get_db`, which closes it after the response.
- `.env` is local. `.env.example` is shared. `pytest` checks the routes without MySQL.

Lab — Ticket ACME-207

Split `main.py` into this layout and add one test.

```
PYTHON/17_FastAPI/labs/L07_structure/
├── broken/      ← Part A. A split project with bugs.
├── starter/     ← Part B. The split files, with TODOs.
└── solution/    ← Open only after the checklist passes.
```

Each folder is a whole project. `cd` into it, copy `.env.example` to `.env`, then run `fastapi dev main.py` and `pytest test_employees.py` from there.

Part A — Fix the broken project (10 min)

`broken/` starts, and it has **3 bugs**:

1. `/docs` has no **employees** section, and `GET /employees/E-1042` is **404**.
2. After you fix 1, `GET /employees/E-1042` is still **404**. Read the path `/docs` shows for that route.
3. `pytest test_employees.py` uses MySQL, not a fresh SQLite file. No `test_acme.db` appears. Depending on your MySQL, the tests fail with `Can't connect` or `Access denied`, or they quietly run against your class database.

Part B — Build the split (15 min)

`starter/` has every file from section 1. `repositories/` and `services/` are complete. Write `get_db`, move your four write routes from Lesson 6 into `routers/employees.py` with `Depends(get_employee_service)`, include the router in `main.py`, and finish the two TODO tests.

Part C — Prove it works (5 min)

- [ ] `fastapi dev main.py` serves `GET /employees/E-1042`.
- [ ] `/docs` groups those routes under **employees**.
- [ ] No route in `routers/employees.py` calls `SessionLocal()` or `commit`.
- [ ] `.env` is not committed. `.env.example` and `.gitignore` are.
- [ ] `pytest test_employees.py` prints `3 passed`.

Then do the same split in your own `acmehr-api` folder.

Homework

1. Push the split project to your own GitHub repo `acmehr-api`.
2. **Transfer drill (15 minutes).** Create `routers/health.py` with its own `APIRouter` and move `GET /health` into it. Remove the route from `main.py` and include the new router instead. The test for `/health` still passes.

Next: Authentication. A login route returns a token, and delete refuses anyone who is not HR.